BEM-CUDA Manual

GPU-Accelerated Boundary Element Method
for Light Scattering

April 2026

Contents

1 Overview **2**

2 Command-Line Reference **2**
- 2.1 Required Parameters . . . 2
- 2.2 Physical Parameters . . . 2
- 2.3 Geometry & Mesh . . . 2
- 2.4 Solver Selection . . . 2
- 2.5 GMRES Parameters (Iterative Solvers) . . . 3
- 2.6 FMM Parameters . . . 3
- 2.7 Preconditioner . . . 3
- 2.8 Orientation Averaging . . . 4
- 2.9 Output . . . 4

3 Working with OBJ Meshes **4**
- 3.1 Preparing the Mesh . . . 4
- 3.2 Mesh Resolution . . . 4
- 3.3 Scaling . . . 5
- 3.4 Subdivision . . . 5
- 3.5 Example: Arbitrary Object . . . 5
- 3.6 GPU Memory Estimates . . . 5

4 Solvers **5**
- 4.1 Dense LU (`-solver dense`) . . . 5
- 4.2 FMM+GMRES (`-solver fmm`) . . . 5
- 4.3 pFFT+GMRES (`-solver pfft`) . . . 5
- 4.4 Surface pFFT (`-solver spfft`) . . . 6

5 Preconditioners **6**
- 5.1 Block-Jacobi + RAS . . . 6

6 Examples **6**

7 Build **7**

8 References **7**

1 Overview

BEM-CUDA solves the electromagnetic scattering problem for dielectric particles using the Boundary Element Method with PMCHWT formulation. The $2N \times 2N$ system:

$$\begin{pmatrix} \eta_e L_{\text{ext}} + \eta_i L_{\text{int}} & -(K_{\text{ext}} + K_{\text{int}}) \\ K_{\text{ext}} + K_{\text{int}} & \dfrac{L_{\text{ext}}}{\eta_e} + \dfrac{L_{\text{int}}}{\eta_i} \end{pmatrix} \begin{pmatrix} J \\ M \end{pmatrix} = \begin{pmatrix} b_J \\ b_M \end{pmatrix}$$

where L, K are single-layer and double-layer BEM operators, $\eta_e = 1$, $\eta_i = 1/|m|$, and m is the complex refractive index.

2 Command-Line Reference

2.1 Required Parameters

Flag	Type	Description
`-ka F`	float	Size parameter $ka = 2\pi a_{\text{eff}}/\lambda$

2.2 Physical Parameters

Flag	Type	Default	Description
`-ri RE IM`	2 floats	1.3116 0	Complex refractive index $m = n_{\text{re}} + i\, n_{\text{im}}$

2.3 Geometry & Mesh

Flag	Type	Default	Description
`-shape TYPE`	string	sphere	Particle shape: `sphere`, `hex`
`-ref N`	int	3	Icosphere refinement level
`-ar F`	float	1.0	Hex prism aspect ratio H/D
`-obj FILE`	string	–	Load mesh from Wavefront OBJ file
`-subdiv N`	int	0	Subdivide OBJ mesh N times ($4^N\times$ triangles)

Mesh sizes by refinement level:

ref	Triangles	RWG (N)	System ($2N$)	Suitable ka
2	320	480	960	1–2
3	1 280	1 920	3 840	2–5
4	5 120	7 680	15 360	5–10
5	20 480	30 720	61 440	10–20
6	81 920	122 880	245 760	20–40

Rule of thumb: ∼10 elements per interior wavelength ($\lambda_{\text{int}}/h \geq 10$). The solver prints mesh resolution diagnostics: `lambda_ext/h` and `lambda_int/h`.

2.4 Solver Selection

Flag	Type	Default	Description

`-solver TYPE`	string	dense	Solver type

Solver	Method	Complexity	Best for
`dense`	Direct LU (cuSOLVER)	$O(N^3)$ time, $O(N^2)$ mem	$N \leq 8000$
`fmm`	Plane-wave MLFMA + GMRES	$O(N \log N)$ matvec	Large N, any shape
`pfft`	Precorrected FFT + GMRES	$O(N \log N)$ matvec	Smooth geometries
`spfft`	Surface pFFT + GMRES	$O(N \log N)$ matvec	Hex prisms
`auto`	Auto-select + auto precond	–	**Recommended**

`auto` selects the solver based on N, ka, and shape, and enables auto-preconditioner.

2.5 GMRES Parameters (Iterative Solvers)

Flag	Type	Default	Description
`-gmres-tol F`	float	10^{-4}	Relative convergence tolerance
`-gmres-restart N`	int	100	Restart parameter m (Krylov subspace size)

GMRES memory: $m \times 2N \times 16$ bytes. Example: $m = 100$, $N = 9216 \rightarrow 60\,\text{MB}$.

Important: GMRES tolerance should not be tighter than FMM accuracy. Without preconditioner, GMRES may fail to converge at ref ≥ 4.

2.6 FMM Parameters

Flag	Type	Default	Description
`-digits N`	int	3	FMM accuracy digits
`-max-leaf N`	int	64	Max particles per octree leaf

2.7 Preconditioner

Flag	Type	Default	Description
`-prec TYPE`	string	none	Preconditioner type
`-prec-r F`	float	2.0	Block-Jacobi cell radius multiplier
`-prec-bs N`	int	1500	Max RWG per block (larger $\rightarrow$ bisected)
`-prec-overlap N`	int	0	RAS overlap layers

Type	Description	Build	Iteration reduction
`none`	No preconditioning	0	–
`diag`	2×2 block-diagonal	$\sim$0 s	Moderate
`blockj`	Block-Jacobi + dense LU	$\sim$30 s	50–90%
`blockj+overlap=1`	RAS overlap	$\sim$150 s	95–99%
`auto`	Auto-select (with `-solver auto`)	Adaptive	Adaptive

Recommended for production (ref ≥ 4, high ka):

```
--prec blockj --prec-r 2.0 --prec-bs 1000 --prec-overlap 1 --gmres-restart 200
```

Creates $\sim$12 blocks with RAS overlap, 14–22 matvecs instead of 700+.

2.8 Orientation Averaging

Flag	Type	Default	Description
`-single`	flag	off	Single orientation, no averaging
`-orient NA NB NG`	3 ints	8 8 1	GL quadrature grid ($\alpha \times \beta \times \gamma$)
`-orient-auto [R]`	opt float	$R = 4$	Auto-compute from ka, $D_{\max}$
`-orient-tol F`	float	0	Adaptive stop tolerance
`-orient-sym B G`	2 ints	1 1	Symmetry: $B = 2$ (mirror), $G = 6$ (C_6)
`-gamma-mirror`	flag	off	σ_v mirror symmetry
`-orient-range I0 I1`	2 ints	–	Compute only orientations $[I_0, I_1)$

`-orient-auto` formula:

$$\delta = \frac{0.69\lambda}{3R\,D_{\max}}, \quad N_\alpha = \lceil 360^\circ/\delta \rceil, \quad N_\beta = \lceil 180^\circ/\delta \rceil$$

2.9 Output

Flag	Type	Default	Description
`-out FILE`	string	result.json	Output JSON file
`-ntheta N`	int	181	Scattering angles (0° to 180°)
`-quad N`	int	7	Triangle quadrature order: 4, 7, 13

Output JSON: `mueller` ($16 \times N_\theta$), `theta`, `ka`, `ri`, `timing`.

3 Working with OBJ Meshes

3.1 Preparing the Mesh

The OBJ file must contain a **closed, manifold, triangulated** surface:

- Triangulated faces only (no quads)
- Closed surface (no holes)
- Manifold edges (each edge shared by exactly 2 triangles)
- Consistent face orientation (normals outward)

Repair tools: MeshLab (Filters → Cleaning), Blender (3D Print Toolkit).

3.2 Mesh Resolution

The solver prints diagnostics:

```
Resolution: h_avg=0.1057, lambda_ext/h=11.9, lambda_int/h=9.1 [OK]
```

λ_{int}/h	Status	Action
≥ 10	OK	Good resolution
6–10	Marginal	5–10% error possible
< 6	WARNING	Use `-subdiv` or reduce ka

3.3 Scaling

The mesh is automatically scaled so $D_{\max}$ matches the size parameter ka. No manual scaling is needed.

3.4 Subdivision

`-subdiv N` performs N rounds of flat subdivision: each triangle $\to$ 4 triangles.

Example: 1200-triangle OBJ + `-subdiv 2` = 19 200 triangles $\approx$ 28 800 RWG.

3.5 Example: Arbitrary Object

```
# Quick test (1 orientation)
bin/bem_cuda --ka 5 --ri 1.3116 0 --obj model.obj --subdiv 1 \
  --solver auto --single --out test.json

# Full orientation averaging
bin/bem_cuda --ka 5 --ri 1.3116 0 --obj model.obj --subdiv 1 \
  --solver auto --orient-auto --out result.json
```

3.6 GPU Memory Estimates

With leaf-to-leaf P2P optimization:

N (RWG)	FMM arrays	Preconditioner	GMRES ($m = 100$)	Total
2 304	~100 MB	~200 MB	15 MB	~0.3 GB
9 216	~170 MB	~1.2 GB	60 MB	~1.4 GB
30 000	~500 MB	~3 GB	200 MB	~3.7 GB
100 000	~1.5 GB	~8 GB	600 MB	~10 GB

RTX 3080 Ti (12 GB): up to ~30 000 RWG comfortably.

4 Solvers

4.1 Dense LU (`-solver dense`)

Direct LU factorization via cuSOLVER. $O(N^3)$ time, $O(N^2)$ memory. Practical for $N \leq 8000$ (ref ≤ 4). No iterative convergence issues.

4.2 FMM+GMRES (`-solver fmm`)

Plane-wave MLFMA with GMRES iterative solver. Matrix-free $O(N \log N)$ matvec. GPU-accelerated P2P, P2M, M2L, L2P kernels.

P2P near-field. Leaf-to-leaf neighbor structure (~1 MB) instead of point-to-point CSR (~6 GB). Each target looks up its leaf, iterates over neighboring leaves, and evaluates $G(\mathbf{r}_i, \mathbf{r}_j)$ for all sources in those leaves.

Two-stream pipeline. P2P and FMM tree traversal run concurrently on separate CUDA streams.

Shared tree. For PMCHWT (two wavenumbers k_{ext}, k_{int}), the octree, P2P structure, and positions are shared. Only k-dependent arrays are duplicated.

4.3 pFFT+GMRES (`-solver pfft`)

Precorrected FFT: 3D Cartesian grid with FFT-based far-field.

4.4 Surface pFFT (`-solver spfft`)

2D FFT per flat face for hex prisms: FP32 C2C, CUDA streams, mixed-radix grids.

5 Preconditioners

Right preconditioning: solve $ZM^{-1}(Mx) = b$.

5.1 Block-Jacobi + RAS

Spatial cell blocks with dense LU. Adaptive bisection for oversized blocks. GPU-accelerated apply with warp-parallel triangular solve.

RAS overlap extends each block with neighboring RWGs:

$$\text{overlap_dist} = \text{overlap_layers} \times 2\,\overline{h}_{\text{extent}}$$

Benchmark (hex $D/L = 0.7$, $m = 1.3116$, RTX 3080 Ti):

ref	ka	Precond	Blocks	Build (s)	Matvecs	Total (s)
3	10	none	–	–	451	85
3	10	blockj	4	1.4	392	115
4	5	blockj+RAS(1)	73	31	22	50
4	**16**	**blockj+RAS(1)**	**12**	**154**	**14**	**184**
4	20	blockj+RAS(1)	4	2432	13	2476

12-block RAS at $ka = 16$: **51× fewer iterations, 13× faster** vs baseline.

6 Examples

```
# 1. Dense LU, sphere, ka=5, single orientation
bin/bem_cuda --ka 5 --ref 3 --ri 1.3116 0 --single

# 2. FMM, hex D/L=0.7, Block-Jacobi+RAS, 64 orientations
bin/bem_cuda --solver fmm --ka 10 --ref 4 --shape hex --ar 1.4286 \
  --prec blockj --prec-overlap 1 --gmres-restart 200 \
  --orient 8 8 1 --out hex_ka10.json

# 3. Auto solver, auto orientations, hex
bin/bem_cuda --solver auto --ka 15 --ref 4 --shape hex --ar 0.7 \
  --orient-auto --out hex_ka15.json

# 4. OBJ mesh with subdivision
bin/bem_cuda --ka 5 --ri 1.5 0.01 --obj statue.obj --subdiv 1 \
  --solver auto --orient-auto --out statue_ka5.json

# 5. Cluster: split 805 orientations across 4 nodes
bin/bem_cuda --solver auto --ka 15 --ref 4 --shape hex --ar 0.7 \
  --orient 35 23 1 --orient-range 0 202 --out node0.json
# ... --orient-range 202 404, 404 606, 606 805

# 6. High-ka + RAS
bin/bem_cuda --solver spfft --shape hex --ar 0.7 --ka 20 --ref 4 \
  --prec blockj --prec-r 2.0 --prec-bs 1000 --prec-overlap 1 \
  --gmres-restart 200 --gmres-tol 1e-4 --orient 45 31 1 \
  --out hex_ka20_r4.json
```

7 Build

```
make -j$(nproc)
```

For CUDA 12.8+ with gcc 15 (too new for nvcc):

```
make -j$(nproc) NVFLAGS="-arch=sm_86 -O3 --use_fast_math \
  -ccbin g++-13 -Xcompiler '-O2 -Wall -std=c++11 -fopenmp' -std=c++11"
```

Set GPU architecture in `Makefile` (default: `sm_86`).

8 References

1. Rao, Wilton, Glisson, "Electromagnetic scattering by surfaces of arbitrary shape," *IEEE TAP*, 1982.
2. Chew, Jin, Michielssen, Song, *Fast and Efficient Algorithms in CEM*, 2001.
3. Graglia, "On the numerical integration of the linear shape functions times the 3-D Green's function," *IEEE TAP*, 1993.
4. Phillips, White, "A precorrected-FFT method for electrostatic analysis," *IEEE TCAD*, 1997.